As far as verifying that this works is concerned, you're going to get some insight into the work the server is doing behind the scenes. Go back to your server window and watch what happens when you refresh the page. The first time you load the page, you should see some SQL that is loading the products like `Product Load (0.2ms) SELECT "products".* FROM "products" ORDER BY "products"."title" ASC`. When you refresh the page again, it will still work, but you *won't* see that SQL run. You *should* see some SQL that Rails runs to check if its cache is outdated, like so: `SELECT COUNT(*) AS "size", MAX("products"."updated_at") AS timestamp FROM "products"`.

If you still aren't convinced, you can check your `log/development.log` file. In there you should see log messages that look like this:

```
Read fragment views/store/index:f6d3d1696e62859f692c4ae9e7980d0f/…
Write fragment views/store/index:f6d3d1696e62859f692c4ae9e7980d0f/…
Read fragment views/store/index:f6d3d1696e62859f692c4ae9e7980d0f/…
Write fragment views/store/index:f6d3d1696e62859f692c4ae9e7980d0f/…
Read fragment views/store/index:f6d3d1696e62859f692c4ae9e7980d0f/…
Write fragment views/store/index:f6d3d1696e62859f692c4ae9e7980d0f/…
Read fragment views/products/2-20170611204944059695/cb43383298…
```

Once you're satisfied that caching is working, turn caching off in development so that further changes to the template will always be visible immediately:

```
depot> bin/rails dev:cache
```

Once again, wait for the server to restart, and verify that changes to the template show up as quickly as you save them.